Nombres:	**Matrículas**:
Danna Nicole Montoya Navarro	2169354
Pablo Sandoval Ventura	2161741
Carlos Emmanuel Oyervides Miguel	2169716

Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño Arquitectural Del Sistema
Fecha: 12/10/2026	

Bibliografia (Formato APA)

Archivo “Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx” proporcionado en la plataforma TEAMS

Archivo “14_CargoBox.docx” proporcionado en la plataforma TEAMS

Objetivo:

Diseñar y documentar la arquitectura de software integral para la empresa de paquetería **CargoBox**, asegurando una separación estricta de responsabilidades entre el dominio, la aplicación y la infraestructura mediante la aplicación práctica de modelado orientado a dominios.

Procedimiento:

CargoBox es una empresa de servicios logísticos y transporte de paquetería orientada a la gestión de envíos a nivel local y nacional. Su modelo operativo abarca desde la recepción inicial del paquete en sucursal y la clasificación en centros de distribución, hasta el despacho en ruta y la ejecución de la "última milla" para la entrega final al destinatario.

A medida que el volumen de operaciones de CargoBox ha crecido, la empresa ha comenzado a enfrentar importantes desafíos operativos derivados de la complejidad de su flujo logístico. Entre los problemas más críticos destacan:

- **Pérdida de trazabilidad en tiempo real:** Dificultad para mantener informado al cliente y a los operadores sobre la ubicación exacta y el estado actual del envío a lo largo de sus 7 transiciones operativas (Recibido, Clasificado, En Tránsito, Centro Destino, En Reparto, Entregado, Devolución).
- **Vulneración de reglas de negocio en campo:** Inconsistencias al aplicar políticas clave, como el límite estricto de dimensiones y peso por paquete (≤ 30 kg y ≤ 1.5 m), el control de intentos de entrega (máximo 2 reintentos antes de devolver o enviar a punto de recolección) y la falta de capturas fehacientes de evidencia digital de recepción (*POD*).
- **Gestión ineficiente de excepciones y retrasos:** Respuestas tardías ante incidencias operativas (como direcciones incorrectas, ausencias o paquetes dañados) y la ausencia de un mecanismo automatizado para alertar sobre el incumplimiento de los Acuerdos de Nivel de Servicio (*SLA*: 8 h para servicio Express y 24 h para Estándar, así como la declaratoria de pérdida a las > 72 h).

Ante este panorama, la dirección técnica de CargoBox determinó la necesidad de modernizar su infraestructura de software mediante una arquitectura sólida, escalable y mantenible. Para responder a estas exigencias operativas, se adopta el patrón de Domain Model respaldado por los principios de Domain-Driven Design (DDD). Este enfoque permite aislar la complejidad logística en un núcleo de dominio cohesivo (estructurado en los agregados Paquete, Traslado, Entrega e Incidencia), garantizando el cumplimiento estricto de las invariantes del

negocio y proporcionando una base transaccional confiable sobre la cual coordinar la operación en tiempo real.

1. Preparar el caso

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Registro, pesaje y generación de guía	Empleado de sucursal	**RN-01:** Cada paquete debe contener una guía única. **RN-02:** El precio depende de dimensiones, peso (<=30 kg), destino y nivel de servicio.	Captura correcta de datos físicos del paquete, cobro de tarifa, asociación a una guía única y registro inicial como recibido.
CU-02	Despacho, transporte y actualización de rastreo	Empleado del centro de distribución/ Repartidor	**RN-03:** El paquete cambia de estado conforme avanza (recibido, clasificado, en tránsito, centro destino, en reparto). **RN-05:** Cada movimiento produce información de rastreo.	Transición fluida de estados operativos, registro de eventos en el historial de rastreo y asignación correcta al repartidor.
CU-03	Entrega exitosa y registro de evidencia POD	Repartidor/ Destinatario	**RN-04:** El estado final es entregado. **RN-06:** Una entrega requiere evidencia de recepción (firma digital o fotografía).	Cierre exitoso del proceso logístico con el cambio de estado a "Entregado" y almacenamiento de la evidencia POD asociada a la guía.
CU-04	Gestión de incidencias y excepciones operativas	Repartidor/ Soporte Técnico/ Supervisor de Calidad	**RN-07:** El sistema debe contemplar incidencias que interrumpan el flujo normal (ausente, segundo intento, dirección incorrecta, paquete dañado, pérdida o rechazo).	Registro formal de la incidencia, activación de flujos alternativos (programación de segundo intento, envío a punto de recolección, corrección de datos en < 4 horas o declaración de pérdida a las >72 horas).

2. Elegir el patrón de lógica de negocio

El patrón principal seleccionado para CargoBox es **Domain Model**. Se justifica debido a que la complejidad del negocio de paquetería radica en la protección estricta de invariantes (como el límite físico de 30 kg, el control estricto de máximo 2 reintentos de entrega, la obligatoriedad de evidencia POD y la gestión automatizada de alertas de retraso de 8h/24h). Al utilizar el *Paquete* como Agregado Raíz, garantizamos que cualquier cambio de estado o evento de rastreo se ejecute de manera controlada y segura.

Tabla comparativa

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Baja, organiza la lógica en procedimientos individuales por cada transacción, lo cual se vuelve inmanejable ante reglas múltiples.	Media, agrupa la lógica orientada a tablas de la base de datos, útil para reglas tabulares pero rígida ante lógicas complejas.	Alta y robusta, encapsula reglas complejas del negocio dentro del dominio.	**RN-01 a RN-07/ Glosario:** Reglas estrictas de negocio, cálculo de tarifas según peso/dimensiones y obligatoriedad de evidencia.
Cambios esperados	Alto riesgo de duplicación ante modificaciones frecuentes en los flujos operativos.	Moderado, soporta cambios a nivel de estructura tabular, pero se acopla fuertemente al esquema de la base de datos.	Excelente adaptabilidad, aísla los cambios lógicos en el núcleo del dominio, facilitando la evolución sin romper las capas externas.	**BRD y BPMN:** El sistema evoluciona constantemente para manejar múltiples excepciones.
Interacción entre conceptos	Limitada, los datos viajan en estructuras planas sin comportamiento enriquecido entre entidades relacionadas.	Orientada a conjuntos de registros de tablas, dificultando modelar relaciones complejas de objetos logísticos.	Nativa y fluida, modela objetos interconectados con identidad y ciclo de vida.	**US-01 a US-17:** Fuerte interacción entre entidades operativas, clientes, sucursales y centros de distribución.

Estados e invariantes	Nulo control automatizado de estados, depende de actualizaciones manuales dispersas en scripts transaccionales.	Control básico basado en campos de tablas relacionales, con alto riesgo de vulnerar invariantes lógicas.	Altamente protegido; el Agregado Raíz (Paquete) controla estrictamente las transiciones de sus 7 estados operativos e invariantes.	**BPMN y Glosario:** Transición estricta por 7 estados operativos (Recibido, Clasificado, En Tránsito, Centro Destino, En Reparto, Entregado, Devolución).
Criterio adicional (Mantenimiento y Auditoría)	Económico y rápido para prototipos pequeños, pero insostenible para sistemas con alta trazabilidad y auditoría.	Moderado rendimiento en consultas masivas, pero ineficiente para auditorías detalladas de eventos y excepciones.	Garantiza trazabilidad total de eventos de dominio, auditoría de SLA y centralización de evidencias requeridas por el negocio.	**NB-01 a NB-08:** Necesidad crítica de trazabilidad absoluta, control de excepciones y centralización de información.

Ventajas, Riesgos y Consecuencias

Alternativa	Ventajas	Riesgos	Consecuencias
Transaction Script	Simplicidad inicial de implementación y rapidez para construir prototipos funcionales básicos sin diseño previo complejo.	Duplicación severa de código, alta vulnerabilidad a inconsistencias de datos y acoplamiento excesivo que vuelve imposible mantener reglas complejas a largo plazo.	Fracaso ante la necesidad de auditoría y trazabilidad exigidas en CargoBox, generando llamadas constantes de clientes por desactualización de estados.
Table Module	Mejor organización que el script al agrupar la lógica por tablas y menor complejidad en operaciones directas sobre conjuntos de datos tabulares.	Las reglas de negocio de CargoBox no son meramente tabulares; dependen de comportamientos de objetos y transiciones de estado complejas que no encajan de forma natural	Limitación severa para manejar las excepciones operativas y los flujos alternativos definidos en el proceso logístico.

		en una estructura de tabla.	
Domain Model	Centraliza y protege todas las reglas del negocio en un núcleo cohesivo, garantiza el cumplimiento de invariantes de estado, facilita las pruebas unitarias y ofrece una trazabilidad impecable alineada con el dominio logístico.	Curva de aprendizaje técnica más alta, sobrecarga inicial de diseño y mayor complejidad conceptual para estructurar el modelo de objetos y los agregados.	Requiere un esfuerzo inicial de arquitectura mayor, pero soluciona de raíz los problemas de visibilidad, control de estados (7 estados operativos) y gestión de excepciones (incidencias, reintentos y POD).

3. Mapa de Dominio

El diagrama modela la arquitectura orientada a dominios (DDD) para el sistema de gestión y trazabilidad de CargoBox, delimitando claramente el contexto, los agregados especializados, los servicios de dominio y los objetos compartidos:

Contexto Delimitado ("Envíos y Trazabilidad"):

Enmarca el núcleo operativo de la empresa, estructurando el dominio en cuatro agregados independientes con límites de consistencia bien definidos, servicios de dominio especializados y componentes de soporte compartidos.

Agregados del Dominio:

- **Agregado Traslado (Raíz):** Acota la operación logística de tránsito y clasificación en los centros de distribución. Contiene el objeto de valor Movimiento (para registrar los cambios de estado en ruta) y emite eventos de dominio relacionados con el despacho, la clasificación y la llegada a centros de destino.
- **Agregado Paquete (Raíz):** Representa el núcleo principal del envío. Gestiona el estado general del paquete, sus datos físicos y las colecciones de eventos de rastreo (EventoRastreo), emitiendo alertas automáticas de retraso, declaraciones de pérdida y devoluciones.
- **Agregado Entrega (Raíz):** Administra de forma independiente la ejecución de la última milla a cargo del repartidor. Controla internamente

los objetos de valor IntentoEntrega (hasta un máximo de 2 reintentos) y EvidenciaRecepcion (POD obligatorio con firma o fotografía), emitiendo los eventos correspondientes al cierre exitoso o agotamiento de intentos.

- **Agregado Incidencia (Raíz):** Acota la gestión de excepciones operativas (como direcciones incorrectas, destinatarios ausentes, paquetes dañados o rechazos) de manera aislada al flujo principal, relacionándose mediante una referencia por guía (GuiaUnica) y controlando su propio ciclo de vida (abierta, escalada, resuelta).

Servicios de Dominio:

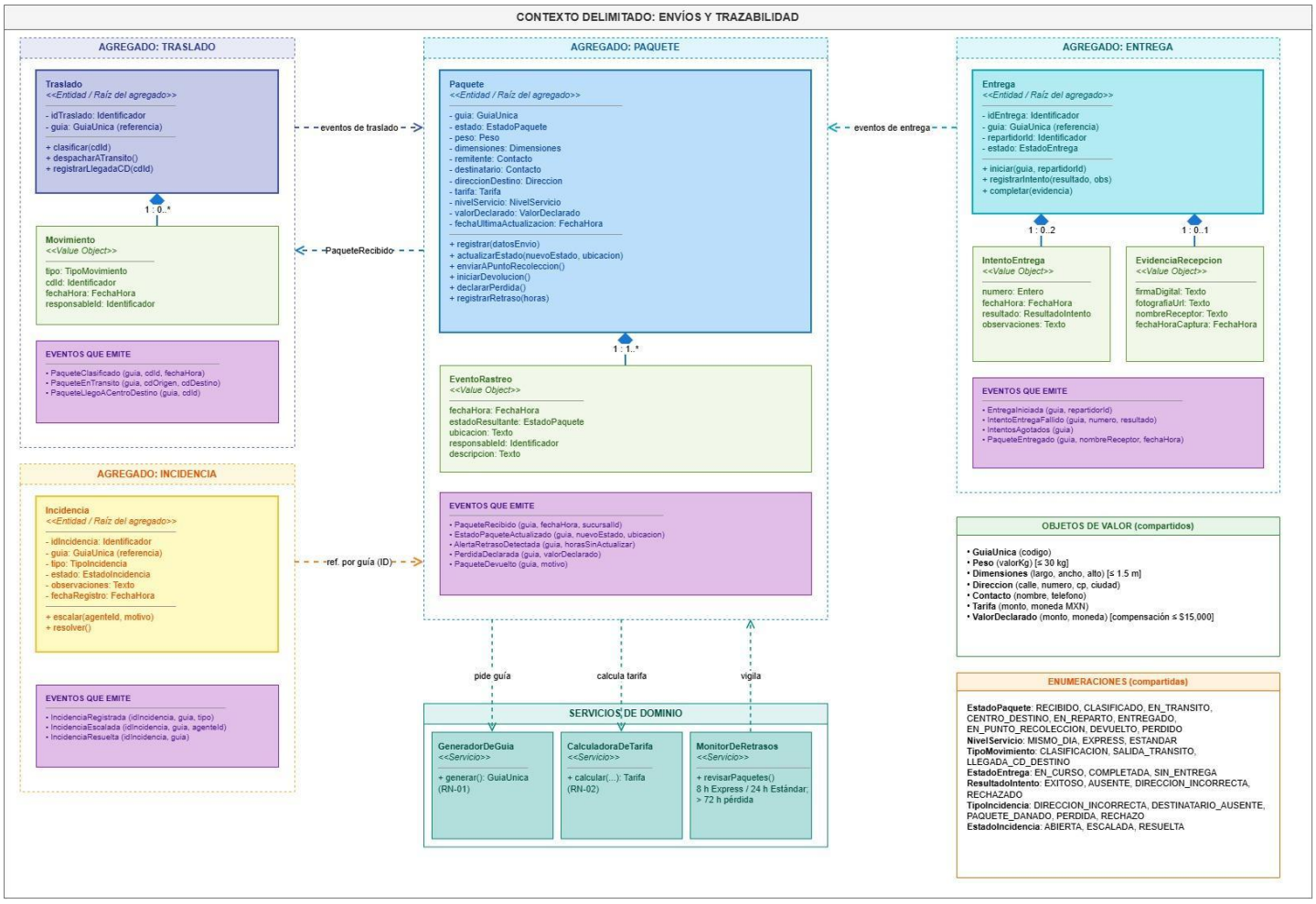
Se definen tres componentes encargados de coordinar la lógica que cruza múltiples entidades:

- **GeneradorDeGuia:** Encargado de emitir el código alfanumérico único para cada envío de acuerdo con la regla de negocio RN-01.
- **CalculadoraDeTarifa:** Aplica las reglas paramétricas de cobro según el peso, las dimensiones, el destino y el nivel de servicio conforme a la regla RN-02.
- **MonitorDeRetrasos:** Ejecuta revisiones automáticas para disparar alertas temporales (8 horas para Express, 24 horas para Estándar) y proponer la declaratoria de pérdida al superarse las 72 horas sin actualización.

Objetos de Valor y Enumeraciones Compartidos:

Se centralizan los elementos inmutables comunes del dominio —tales como GuiaUnica, Peso ($\leq 30\ kg$), Dimensiones ($\leq 1.5\ m$), Direccion, Contacto, Tarifa y ValorDeclarado (con compensación máxima de $\$15{,}000\ MXN$)— junto con las enumeraciones tipadas de estados y resultados que garantizan la consistencia semántica en todo el sistema.

Reporte


CONTEXTO DELIMITADO: ENVÍOS Y TRAZABILIDAD
AGREGADO: TRASLADO
Traslado
<<Entidad / Raíz del agregado>>
- idTraslado: Identificador
- guia: GuiaUnica (referencia)
+ clasificar(cdId)
+ despacharATransito()
+ registrarLlegadaCD(cdId)
1 : 0..*
Movimiento
<<Value Object>>
tipo: TipoMovimiento
cdId: Identificador
fechaHora: FechaHora
responsableId: Identificador
EVENTOS QUE EMITE
• PaqueteClasificado (guia, cdId, fechaHora)
• PaqueteEnTransito (guia, cdOrigen, cdDestino)
• PaqueteLlegoACentroDestino (guia, cdId)
AGREGADO: INCIDENCIA
Incidencia
<<Entidad / Raíz del agregado>>
- idIncidencia: Identificador
- guia: GuiaUnica (referencia)
- tipo: TipoIncidencia
- estado: EstadoIncidencia
- observaciones: Texto
- fechaRegistro: FechaHora
+ escalar(agenteId, motivo)
+ resolver()
EVENTOS QUE EMITE
• IncidenciaRegistrada (idIncidencia, guia, tipo)
• IncidenciaEscalada (idIncidencia, guia, agenteId)
• IncidenciaResuelta (idIncidencia, guia)
eventos de traslado
PaqueteRecibido
ref. por guía (ID)
AGREGADO: PAQUETE
Paquete
<<Entidad / Raíz del agregado>>
- guia: GuiaUnica
- estado: EstadoPaquete
- peso: Peso
- dimensiones: Dimensiones
- remitente: Contacto
- destinatario: Contacto
- direccionDestino: Direccion
- tarifa: Tarifa
- nivelServicio: NivelServicio
- valorDeclarado: ValorDeclarado
- fechaUltimaActualizacion: FechaHora
+ registrar(datosEnvio)
+ actualizarEstado(nuevoEstado, ubicacion)
+ enviarAPuntoRecoleccion()
+ iniciarDevolucion()
+ declararPerdida()
+ registrarRetraso(horas)
1 : 1..*
EventoRastreo
<<Value Object>>
fechaHora: FechaHora
estadoResultante: EstadoPaquete
ubicacion: Texto
responsableId: Identificador
descripcion: Texto
EVENTOS QUE EMITE
• PaqueteRecibido (guia, fechaHora, sucursalId)
• EstadoPaqueteActualizado (guia, nuevoEstado, ubicacion)
• AlertaRetrasoDetectada (guia, horasSinActualizar)
• PerdidaDeclarada (guia, valorDeclarado)
• PaqueteDevuelto (guia, motivo)
eventos de entrega
AGREGADO: ENTREGA
Entrega
<<Entidad / Raíz del agregado>>
- idEntrega: Identificador
- guia: GuiaUnica (referencia)
- repartidorId: Identificador
- estado: EstadoEntrega
+ iniciar(guia, repartidorId)
+ registrarIntento(resultado, obs)
+ completar(evidencia)
1 : 0..2
1 : 0..1
IntentoEntrega
<<Value Object>>
numero: Entero
fechaHora: FechaHora
resultado: ResultadoIntento
observaciones: Texto
EvidenciaRecepcion
<<Value Object>>
firmaDigital: Texto
fotografiaUrl: Texto
nombreReceptor: Texto
fechaHoraCaptura: FechaHora
EVENTOS QUE EMITE
• EntregaIniciada (guia, repartidorId)
• IntentoEntregaFallido (guia, numero, resultado)
• IntentosAgotados (guia)
• PaqueteEntregado (guia, nombreReceptor, fechaHora)
OBJETOS DE VALOR (compartidos)
• GuiaUnica (codigo)
• Peso (valorKg) [≤ 30 kg]
• Dimensiones (largo, ancho, alto) [≤ 1.5 m]
• Direccion (calle, numero, cp, ciudad)
• Contacto (nombre, telefono)
• Tarifa (monto, moneda MXN)
• ValorDeclarado (monto, moneda) [compensación ≤ $15,000]
ENUMERACIONES (compartidas)
EstadoPaquete: RECIBIDO, CLASIFICADO, EN_TRANSITO, CENTRO_DESTINO, EN_REPARTO, ENTREGADO, EN_PUNTO_RECOLECCION, DEVUELTO, PERDIDO
NivelServicio: MISMO_DIA, EXPRESS, ESTANDAR
TipoMovimiento: CLASIFICACION, SALIDA_TRANSITO, LLEGADA_CD_DESTINO
EstadoEntrega: EN_CURSO, COMPLETADA, SIN_ENTREGA
ResultadoIntento: EXITOSO, AUSENTE, DIRECCION_INCORRECTA, RECHAZADO
TipoIncidencia: DIRECCION_INCORRECTA, DESTINATARIO_AUSENTE, PAQUETE_DANADO, PERDIDA, RECHAZO
EstadoIncidencia: ABIERTA, ESCALADA, RESUELTA
pide guía
calcula tarifa
vigila
SERVICIOS DE DOMINIO
GeneradorDeGuia
<<Servicio>>
+ generar(): GuiaUnica (RN-01)
CalculadoraDeTarifa
<<Servicio>>
+ calcular(...): Tarifa (RN-02)
MonitorDeRetrasos
<<Servicio>>
+ revisarPaquetes()
8 h Express / 24 h Estándar; > 72 h pérdida

Fichas de Aggregates

Paquete

Raíz	Paquete (identidad: GuiaUnica). Contiene EventoRastreo, su historial.
Responsabilidades	Registrar el envío y asignar guía y tarifa, apoyándose en los servicios. Mantener el estado y el historial. Moverlo a punto de recolección, devolución o pérdida.
Invariantes	guía única e inmutable (RN-01). Peso ≤ 30 kg y lados ≤ 1.5 m. El estado avanza en el orden de RN-04. Cada cambio de estado agrega un EventoRastreo (RN-05). Solo pasa a ENTREGADO al recibir PaqueteEntregado.
Eventos	PaqueteRecibido, EstadoPaqueteActualizado, AlertaRetrasoDetectada, PerdidaDeclarada, PaqueteDevuelto.

Traslado

Raíz	Traslado (idTraslado), con referencia a la guía. Contiene Movimiento. Nace al recibirse PaqueteRecibido.
Responsabilidades	Registrar clasificación, salida a tránsito y llegada al centro destino (US-04 a US-06).
Invariantes	Los movimientos van en orden (clasificado, en tránsito, centro destino). No hay llegada sin salida. Todo movimiento tiene centro y responsable.
Eventos	PaqueteClasificado, PaqueteEnTransito, PaqueteLlegoACentroDestino.

Entrega

Raíz	Entrega (idEntrega), con referencia a la guía. Contiene IntentoEntrega y EvidenciaRecepcion. Nace cuando el repartidor recibe el paquete para reparto (US-07).
Responsabilidades	Iniciar el reparto, registrar intentos y completar con evidencia (US-07 y US-09 a US-12).
Invariantes	Máximo 2 intentos. El segundo solo existe tras un primer fallo. Una entrega exitosa exige evidencia (RN-06). No se agregan intentos tras completar. Tras el segundo fallo se emite IntentosAgotados.
Eventos	EntregaIniciada, IntentoEntregaFallido, IntentosAgotados, PaqueteEntregado.

Incidencia

Raíz	Incidencia (idIncidencia), con referencia a la guía.
Responsabilidades	Registrar la incidencia (dirección incorrecta, ausente, dañado, pérdida o rechazo), escalarla y resolverla. Es lo que consulta Atención al Cliente.
Invariantes	Toda incidencia tiene guía y tipo. Solo se escala si está Abierta. Solo se resuelve una vez. El escalamiento guarda agente y motivo. Nunca se borra.
Eventos	IncidenciaRegistrada, IncidenciaEscalada, IncidenciaResuelta.

4. Service Layers

El service layer de CargoBox orquesta los casos de uso sobre los agregados Paquete, Traslado, Entrega e Incidencia: carga el agregado por su guía o identificador, invoca su comportamiento, lo persiste y publica los eventos de dominio. No contiene reglas de negocio; estas permanecen encapsuladas en los agregados, sus entidades y sus objetos de valor.

Tabla de Service Layer

Servicio	Operación	Agregado (repositorio)	Reglas que valida el dominio	Evento
ServicioDe Recepcion	registrarEnvio (Command RegistrarEnvio)	Paquete (Paquete Repository)	RN-01 guía única (GeneradorDeGuia, VerificadorDeGuia); RN-02 tarifa (CalculadoraDe Tarifa y PoliticaDe Tarifa); peso ≤ 30 kg; lados ≤ 1.5 m; valor declarado ≤ $15,000	Paquete Recibido
ServicioDe Traslado	abrirTraslado (reacciona a Paquete Recibido)	Traslado (Traslado Repository)	Nace al recibirse PaqueteRecibido; consistencia eventual	—
	clasificar Paquete(guia, cdId)	Traslado	Movimientos en orden; todo movimiento tiene centro y responsable	Paquete Clasificado
	despachar ATransito(guia)	Traslado	Solo tras la clasificación	PaqueteEn Transito

	registrar Llegada ACentro Destino(guia, cdId)	Traslado	No hay llegada sin salida	Paquete Llego ACentro Destino
ServicioDe Entrega	iniciar Reparto(guia, repartidorId)	Entrega (Entrega Repository)	Nace al recibir el repartidor el paquete (US-07); usa ConsultaDe DisponibilidadPara Reparto	Entrega Iniciada
	registrar Intento Fallido(guia, resultado, obs)	Entrega	Máximo 2 intentos; el segundo solo existe tras un primer fallo	Intento Entrega Fallido; Intentos Agotados (tras el 2.º)
	completar Entrega (Command Completar Entrega)	Entrega	RN-06 evidencia obligatoria (firma o fotografía); no se agregan intentos tras completar	Paquete Entregado
ServicioDe ExcepcionesDe Paquete	registrar Incidencia(guia, tipo, obs)	Incidencia (Incidencia Repository)	Toda incidencia tiene guía y tipo (RN-07)	Incidencia Registrada
	escalar Incidencia (Command Escalar Incidencia)	Incidencia	Solo se escala si está Abierta; guarda agente y motivo	Incidencia Escalada

	resolver Incidencia(id Incidencia)	Incidencia	Solo se resuelve una vez; nunca se borra	Incidencia Resuelta
	enviarAPunto Recoleccion(g uia) (reacciona a Intentos Agotados)	Paquete (Paquete Repository)	Consistencia eventual	Estado Paquete Actualizad o
	iniciar Devolucion(gu ia)	Paquete	Transición válida; cada cambio de estado agrega un EventoRastreo (RN-05)	Paquete Devuelto
ServicioDe VigilanciaDe Retrasos	detectar Retrasos() (job programado)	Paquete (ConsultaDe PaquetesSin Actualizacion, Reloj)	PoliticaDeAlerta por nivel (Express 8 h, Estándar 24 h; Mismo Día pendiente); una transacción por paquete	Alerta Retraso Detectada
	declarar Perdida(guia)	Paquete	> 72 h sin actualización; compensación ≤ $15,000	Perdida Declarada

5. Separar persistencia y transacciones

Tabla de Repositorios

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
PaqueteRepository	Recuperar el paquete por su guía para aplicarle un cambio (recepción, estado, pérdida, devolución, retraso) y guardarlo con su historial	obtenerPorGuia(guia), guardar(paquete)	Agregado Paquete con sus EventoRastreo
TrasladoRepository	Recuperar el traslado de un paquete para registrar su siguiente movimiento	siguienteId(), obtenerPorGuia(guia), guardar(traslado)	Traslado con sus Movimiento
EntregaRepository	Recuperar la entrega del paquete para registrar intentos o completarla	siguienteId(), obtenerPorGuia(guia), guardar(entrega)	Entrega con sus IntentoEntrega y su EvidenciaRecepcion
IncidenciaRepository	Recuperar una incidencia para escalarla o resolverla	siguienteId(), obtenerPorId(idIncidencia), guardar(incidencia)	Incidencia

Unit of Work

Operación	Se completa junto (misma transacción)	Queda fuera (consistencia eventual)
registrarEnvio	Paquete con su primer EventoRastreo y el evento PaqueteRecibido	Abrir el Traslado
clasificarPaquete, despacharATransito, registrarLlegadaACentroDestino	Traslado con su Movimiento y su evento	El estado del Paquete
completarEntrega	Entrega con evidencia y el evento PaqueteEntregado	El estado ENTREGADO del Paquete
registrarIntentoFallido	Entrega con el intento, IntentoEntregaFallido y, si es el segundo, IntentosAgotados	Mover el Paquete a punto de recolección
escalarIncidencia	Incidencia y IncidenciaEscalada	Nada
Manejador de PaqueteEntregado	Paquete con estado y EventoRastreo, más marcar el evento como procesado	Nada
detectarRetrasos	Una transacción por paquete	Los demás paquetes

Qué ocurre ante un error:

Error	Resultado
El dominio rechaza la operación (por ejemplo, un tercer intento de entrega)	No se guarda nada y se devuelve un error de negocio.
Falla el guardado	Se revierte todo, incluidos los eventos pendientes. No sale ningún evento.
Conflicto de versión	Se revierte y se reintenta una vez. Si persiste, devuelve ConflictoDeConcurrencia.

Falla la publicación después de confirmar	El evento sigue en la bandeja de salida y se reintenta.
Falla un manejador	Se revierte solo su transacción y se reintenta. Tras varios fallos pasa a una bandeja de errores con alerta. Mientras tanto el Paquete queda atrasado (consistencia eventual).

6. Commands and Queries

Para esta etapa se aplicó CQRS básico, con el fin de separar claramente las operaciones que cambian el estado del sistema de las que solo consultan información. No se crearon bases de datos separadas: tanto los comandos como las consultas trabajan sobre la misma base, pero cada uno con un modelo y un camino distintos. Cada operación se expresa como una solicitud con un nombre del negocio y es atendida por un Handler propio, que declara explícitamente las dependencias que necesita.

Se seleccionaron tres Commands que representan cambios de estado relevantes del negocio. RegistrarEnvio da de alta el paquete, genera su guía única y calcula su tarifa. CompletarEntrega cierra el reparto y exige la evidencia de recepción. EscalarIncidencia pasa un caso a un responsable cuando Atención al Cliente no puede resolverlo. En los tres, el Handler solo coordina: carga el agregado, invoca su comportamiento, guarda dentro de una unidad de trabajo y devuelve una confirmación mínima, sin exponer el agregado. Las reglas del negocio, como los límites de peso, la evidencia obligatoria o el estado desde el cual se puede escalar, permanecen en el dominio.

Se seleccionaron además tres Queries que responden a necesidades de información de los usuarios. ConsultarRastreo muestra al remitente el estado de su envío y su historial de movimientos. ConsultarIncidenciasDePaquete entrega a Atención al Cliente la lista de incidencias de una guía. ListarPaquetesConAlerta permite revisar los paquetes que llevan demasiado tiempo sin actualización. Estas consultas no cargan agregados ni invocan comportamiento del dominio: leen directamente los datos y los devuelven en DTOs diseñados para lectura, con estados en texto legible, campos calculados como las horas sin actualizar y

solo los datos que cada consumidor necesita, por lo que no reutilizan el modelo de escritura.

No todo el caso requiere esta separación. Los comandos de traslado (clasificar, despachar y registrar llegada) solo cambian estado y su historial ya se consulta a través del rastreo, por lo que no necesitan una consulta propia. Tampoco hace falta un almacén de lectura independiente ni una infraestructura de mensajería compleja, ya que la separación se mantiene en el nivel de las responsabilidades del código y no en el de la base de datos.

Tipo	Nombre	Datos de entrada	Handler (dependencias)	Resultado	Justificación
Command	RegistrarEnvio	remitente, destinatario, dirección destino, peso, dimensiones, nivel de servicio, valor declarado, sucursalId	RegistrarEnvioHandler (PaqueteRepository, GeneradorDeGuia, CalculadoraDeTarifa, UnidadDeTrabajo)	guía y tarifa	Crea el Paquete. Devuelve solo lo que el cliente necesita, no el agregado.
Command	CompletarEntrega	guía, firma o fotografía, nombre del receptor	CompletarEntregaHandler (EntregaRepository, UnidadDeTrabajo)	confirmación (guía y fecha)	Cambia el estado y exige evidencia. El dominio protege la regla.
Command	EscalarIncidencia	idIncidencia, agenteId, motivo	EscalarIncidenciaHandler (IncidenciaRepository, UnidadDeTrabajo)	confirmación (idIncidencia y estado)	Cambia el estado de la incidencia.

Query	ConsultarRastreo	guía	ConsultarRastreoHandler (ConsultaDeRastreo)	RastreoDeEnvioDto	Solo lectura. No carga el agregado Paquete (US-08).
Query	ConsultarIncidenciasDePaquete	guía	ConsultarIncidenciasHandler (ConsultaDeIncidencias)	lista de IncidenciaResumenDto	Atención al Cliente necesita una lista, no un agregado (US-15).
Query	ListarPaquetesConAlerta	nivel de servicio (opcional)	ListarAlertasHandler (ConsultaDeAlertas)	lista de PaqueteConAlertaDto	Vista operativa para revisar casos (US-17).

7. Revisión con SOLID

Antes de cerrar los diagramas revisamos la propuesta completa con los cinco principios SOLID, con el objetivo de detectar clases que hicieran demasiado, dependencias innecesarias o reglas que obligaran a modificar código existente.

Obtuvimos siete hallazgos: dos confirman buenas decisiones y cinco señalaron correcciones que aplicamos.

- **S (responsabilidad única), confirmación y corrección:** confirmamos que dividir el dominio en cuatro agregados (Paquete, Traslado, Entrega e Incidencia) deja a cada uno con una sola razón para cambiar, en lugar de un Paquete que concentrara intentos, evidencia e incidencias. Corregimos ServicioDePaquetes, que mezclaba recepción, excepciones y vigilancia de retrasos: ahora son ServicioDeRecepcion, ServicioDeExcepcionesDePaquete y ServicioDeVigilanciaDeRetrasos.
- **O (abierto/cerrado), corrección:** CalculadoraDeTarifa y MonitorDeRetrasos dependían del nivel de servicio (Mismo Día, Express, Estándar), de modo que cada nivel nuevo obligaba a editar condiciones. Ahora cada nivel tiene su PoliticaDeTarifa y su PoliticaDeAlerta, y agregar uno es sumar una política sin tocar los servicios. Esto también aísla en una sola clase la duda pendiente del umbral de Mismo Día.

- **L (sustitución de Liskov), corrección:** definimos un contrato común para los repositorios (nunca devuelven nulo, lanzan el error de negocio correspondiente y guardar lanza ConflictoDeConcurrencia) y pruebas de contrato que corren contra la implementación SQL y la de memoria, de modo que cambiar de implementación no altere el comportamiento.
- **I (segregación de interfaces), corrección:** PaqueteRepository quedó solo con obtenerPorGuia y guardar. La verificación de guías repetidas vive en VerificadorDeGuia y la búsqueda de paquetes sin actualizar en ConsultaDePaquetesSinActualizacion, para que cada consumidor dependa solo de lo que usa.
- **D (inversión de dependencias), confirmación y corrección:** confirmamos que servicios y handlers dependen solo de abstracciones (repositorios, UnidadDeTrabajo, PublicadorDeEventos y Reloj) y que el SQL queda en la infraestructura. Corregimos que Entrega recibía el estado del Paquete, un tipo de otro agregado: ahora ConsultaDeDisponibilidadParaReparto devuelve un valor propio de Entrega.

Dejamos documentado un riesgo: el agregado Paquete sigue concentrando los datos del envío, su estado y su historial de rastreo, y su estado depende de eventos de Traslado y Entrega. No lo dividimos más porque separar el estado del historial rompería la regla que protege (cada cambio de estado agrega un evento de rastreo); lo mitigamos con value objects (GuiaUnica, Peso, Dimensiones, Tarifa), con los servicios de dominio GeneradorDeGuia y CalculadoraDeTarifa y con manejadores de eventos idempotentes. Además, Traslado es el agregado con menos reglas propias, por lo que su separación queda sujeta a revisión.

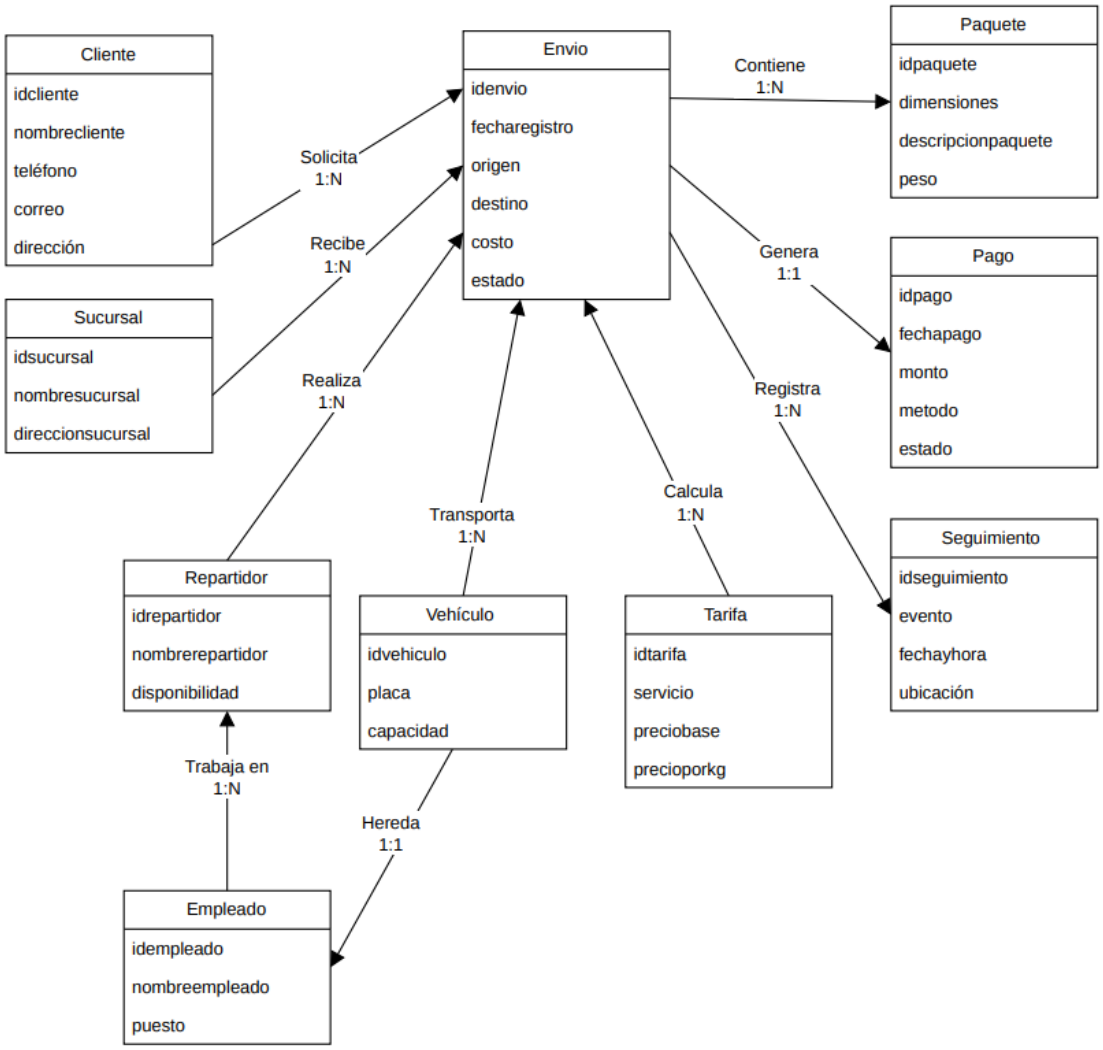
Tabla de revisión SOLID

Principio	Pregunta propuesta	Respuesta con evidencia de nuestro diseño
S	¿Cada clase o servicio tiene una razón principal para cambiar?	Sí: cuatro agregados con reglas propias y servicios separados por actor (ServicioDeRecepcion, ServicioDeExcepcionesDePaquete, ServicioDeVigilanciaDeRetrasos). Vigilar Paquete, que concentra datos, estado e historial.
O	¿Se puede agregar una variante de regla sin editar una cadena de condiciones?	Riesgo con los niveles de servicio: tarifas y umbrales de alerta cambian por nivel. Se resolvió con PoliticaDeTarifa y PoliticaDeAlerta por nivel; Mismo Día solo requiere una política nueva.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento?	Los repositorios SQL y en memoria deben comportarse igual, y el contrato no estaba escrito. Ahora lo fija la interfaz (nunca nulo, error de negocio, control de versión) y se verifica con pruebas de contrato.
I	¿Cada consumidor depende solo de lo que usa?	PaqueteRepository iba a mezclar escritura, verificación de guías y búsqueda de retrasos, pero cada consumidor necesita una sola. Se separó en PaqueteRepository, VerificadorDeGuia y ConsultaDePaquetesSinActualizacion.
D	¿Aplicación y dominio dependen de abstracciones?	Sí: solo interfaces (repositorios, UnidadDeTrabajo, PublicadorDeEventos, Reloj). No hay SQL en el modelo. Se corrigió que Entrega dependiera del estado del Paquete.

8. Diagrama de Clases

Representa la estructura del sistema de gestión de paquetería mediante la identificación de las entidades principales que intervienen en el proceso de

envío, transporte, pago y seguimiento de paquetes. Cada clase contiene atributos que permiten almacenar información relevante para las operaciones de la empresa, mientras que las relaciones establecen cómo interactúan las distintas entidades para completar el ciclo de vida de un envío. El envío representa la operación principal del negocio y conecta a los clientes, las sucursales, los repartidores, los vehículos, las tarifas, los pagos, los paquetes y los registros de seguimiento. Esta organización permite centralizar la información de cada operación logística y mantener una relación entre los datos administrativos y las actividades físicas necesarias para transportar una mercancía desde su origen hasta su destino. La clase Envío, que actúa como vínculo entre las entidades administrativas, económicas y logísticas. Cliente identifica a quien solicita el servicio; Paquete describe la mercancía; Sucursal representa las instalaciones participantes; Repartidor y Vehículo intervienen en el transporte; Tarifa proporciona las reglas de cálculo del costo; Pago registra la transacción económica; y Seguimiento conserva los acontecimientos del proceso. Las multiplicidades expresadas en el diagrama indican cuántas instancias de una clase pueden relacionarse con instancias de otra.



9. Diagramas de Secuencia

Diagrama 1

Utilizamos únicamente cuatro participantes principales: Cliente, Sistema, Envío y Repartidor. Aunque el repartidor no interviene directamente en el registro inicial, forma parte del modelo general de CargoBox. Centralizamos en la clase Sistema las operaciones relacionadas con la validación de datos, el cálculo de tarifas, el procesamiento de pagos y el registro de solicitudes. La clase Envió será responsable de representar la operación logística creada, conservar sus datos principales y mantener su estado. El cliente interactúa directamente con el sistema para proporcionar los datos del paquete, consultar la cotización y

aceptar o rechazar la operación. Una vez que los datos sean validados y el pago sea aprobado, el sistema creará el envío y generará su número de guía. Si los datos son incorrectos, el sistema notificará los errores de validación. Si el pago es rechazado, informará el resultado y evitará que la operación se confirme como un envío pagado.

Diagrama 1: Registro y creación de un envío

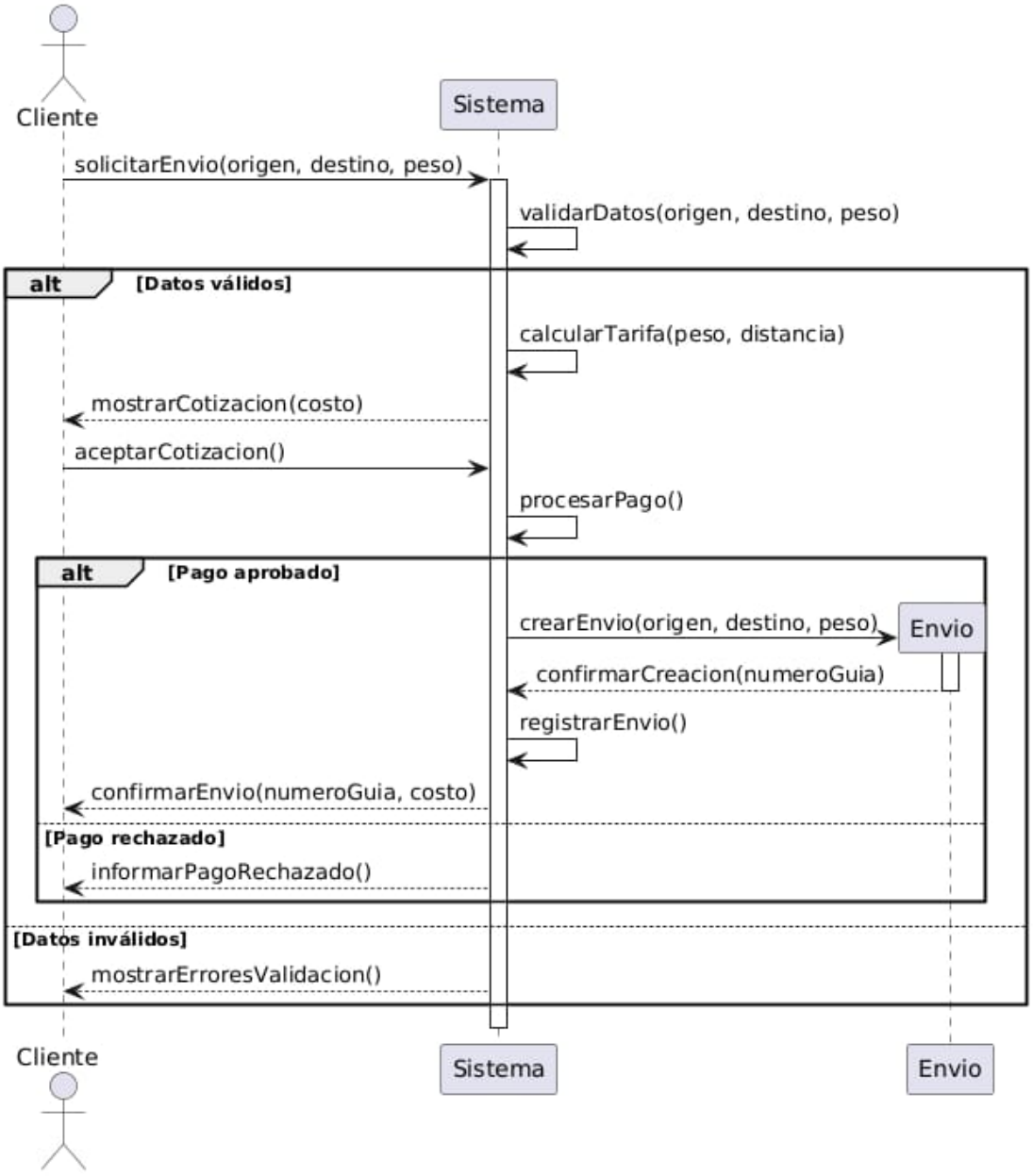
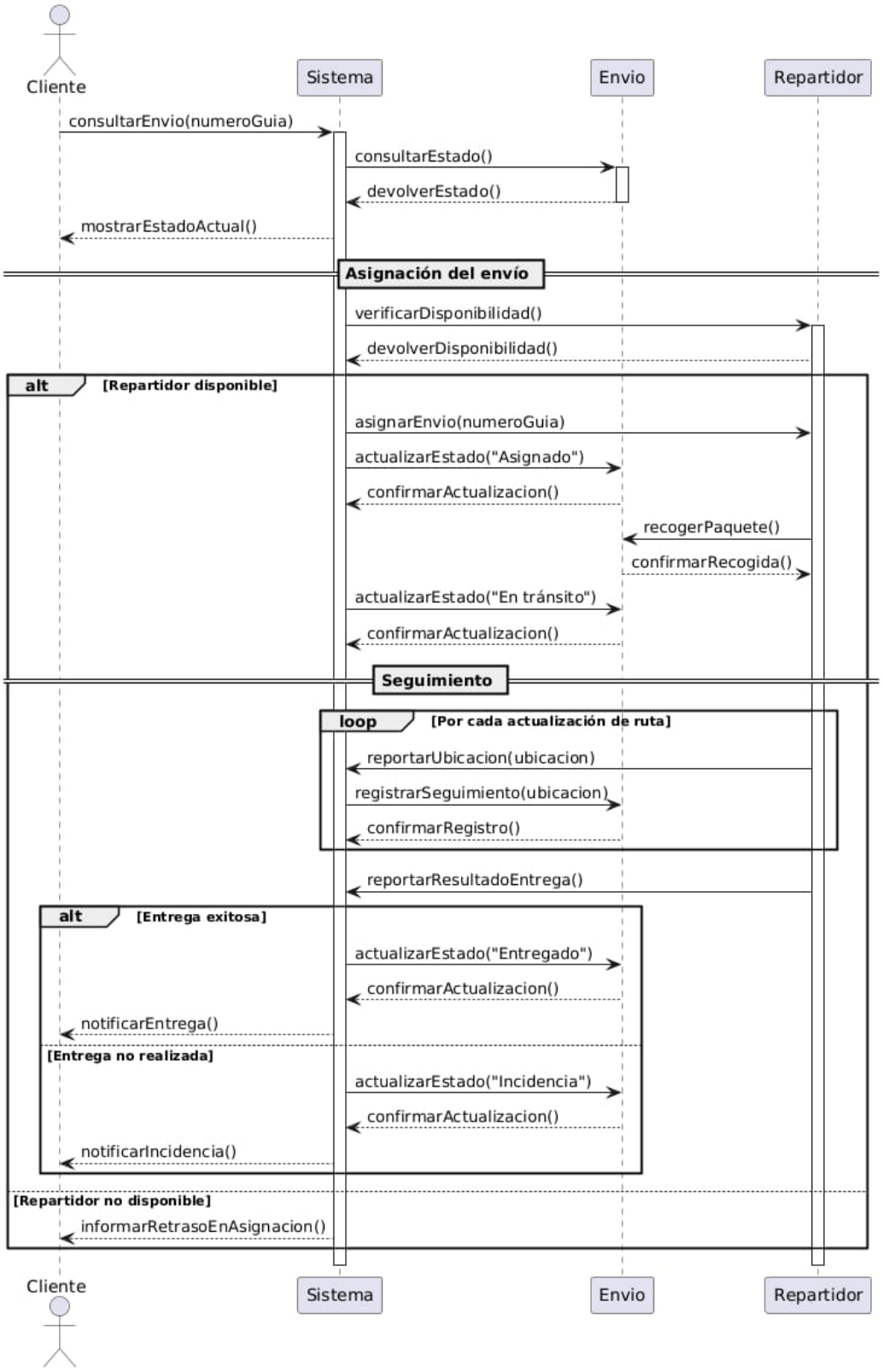


Diagrama 2

El sistema también debe permitir que los clientes consulten el estado de sus paquetes. Para ello, necesita mantener información actualizada sobre las etapas del proceso logístico y registrar las incidencias que impidan completar una entrega. El problema arquitectónico consiste en definir cómo coordinar estas actividades utilizando el modelo simplificado de cuatro participantes. El proceso comienza cuando el sistema recibe una solicitud de consulta o inicia la gestión de un envío registrado. Posteriormente, verifica la disponibilidad del repartidor y realiza la asignación correspondiente. Cuando el paquete es recogido, el sistema actualiza su estado a «En tránsito». Durante el recorrido, el repartidor comunica las actualizaciones de ubicación y el sistema registra los acontecimientos asociados al envío. Al finalizar el transporte, el repartidor informa el resultado de la entrega. Si la operación se completa, el sistema actualiza el estado a «Entregado» y notifica al cliente. Si la entrega no puede realizarse, registra una incidencia y comunica la situación al cliente. Si no existe un repartidor disponible, el sistema informa que la asignación no pudo completarse.

Diagrama 2: Transporte, seguimiento y entrega



10. Architecture Decision Records

Un ADR registra una decisión de arquitectura con su contexto, las alternativas que se consideraron, lo que se decidió y las consecuencias. Escribimos cuatro, cada uno con su estado, contexto, alternativas, decisión, consecuencias y trazabilidad hacia necesidades, reglas, historias y casos de uso.

Resumen de los ADR:

- **ADR-01, registro, cotización y creación del envío:** Domain Model respaldado por una Capa de Aplicación. Consecuencia aceptada: mayor curva de aprendizaje inicial.
- **ADR-02, gestión de transporte, seguimiento y entrega:** flujo coordinado de estados y bloques de control, en lugar de actualizar directamente la base de datos. Consecuencia aceptada: alto volumen de escrituras en base de datos.
- **ADR-03, gestión de incidencias, reintentos y evidencia de entrega:** límite de 2 intentos y evidencia digital (POD) obligatoria. Consecuencia aceptada: dependencia de la conectividad móvil del repartidor.
- **ADR-04, monitoreo automatizado de retrasos y pérdidas:** procesos automáticos en segundo plano con separación de consultas (CQRS básico). Consecuencia aceptada: mayor consumo de recursos por tareas en segundo plano.

ADR-01, registro, cotización y creación del envío

Campo	Contenido
Título	Registro, cotización y creación del envío
Estado	Aceptado
Contexto	Se requiere un mecanismo centralizado para procesar solicitudes de envío, validar datos, calcular tarifas según peso y dimensiones, procesar pagos y generar una guía única sin inconsistencias.
Alternativas	1. Transaction Script. 2. Table Module. 3. Domain Model (elegida).
Decisión	Adoptar Domain Model respaldado por una Capa de Aplicación, donde la clase principal y el agregado gestionan validaciones, tarifas y confirmación de pago antes de emitir la guía.
Consecuencias	**Beneficios:** alta cohesión y transparencia de costos. **Riesgos:** mayor curva de aprendizaje inicial.
Trazabilidad	CU-01, reglas de validación y cotización, Diagrama de Secuencia 1.

ADR-02, gestión de transporte, seguimiento y entrega

Campo	Contenido
Título	Gestión de transporte, seguimiento y entrega de envíos
Estado	Aceptado
Contexto	Durante la distribución logística, se necesita coordinar la disponibilidad de repartidores, actualizar estados a «En tránsito», registrar ubicaciones y gestionar el resultado del despacho.

Alternativas	1. Actualización directa de BD. 2. Coordinación mediante flujo estructurado de estados y bloques de control (elegida).
Decisión	Implementar un flujo coordinado que verifica la disponibilidad del repartidor, controla las transiciones de estado del envío y registra de forma iterativa los eventos de ruta.
Consecuencias	**Beneficios:** trazabilidad de punta a punta. **Riesgos:** alto volumen de escrituras en base de datos.
Trazabilidad	CU-02 y CU-03, BPMN de distribución, Diagrama de Secuencia 2.

ADR-03, gestión de incidencias, reintentos y evidencia de entrega

Campo	Contenido
Título	Gestión de incidencias, reintentos y evidencia de entrega
Estado	Aceptado
Contexto	Las entregas físicas experimentan anomalías (ausencia del destinatario o daños). Se requiere un límite estricto de reintentos y una verificación fehaciente para evitar disputas.
Alternativas	1. Intentos indefinidos sin validación. 2. Límite de 2 intentos y captura obligatoria de evidencia digital POD (elegida).
Decisión	Incorporar reglas de dominio que limitan a un máximo de 2 los intentos de entrega fallidos antes de ir a punto de recolección, exigiendo evidencia de recepción (POD) para marcar como «Entregado».
Consecuencias	**Beneficios:** menos aclaraciones y mayor control de calidad en campo.

	Riesgos: dependencia de la conectividad móvil del repartidor.
Trazabilidad	Regla de Negocio RN-06 (POD), Service Layer (EntregaService e IncidenciaService).

ADR-04, monitoreo automatizado de retrasos y pérdidas

Campo	Contenido
Título	Monitoreo automatizado de retrasos y pérdidas
Estado	Aceptado
Contexto	La empresa necesita auditar proactivamente el cumplimiento de los tiempos de entrega (SLA) y declarar la pérdida formal de paquetes tras superar un umbral crítico sin actualización.
Alternativas	1. Auditoría manual por operadores. 2. Procesos automatizados en segundo plano con separación de consultas (elegida).
Decisión	Diseñar un servicio de monitoreo automático que evalúa las horas sin actualización según el nivel de servicio (CQRS básico) y declara la pérdida formal al cumplirse 72 horas.
Consecuencias	**Beneficios:** detección temprana de anomalías y compensaciones transparentes. **Riesgos:** mayor consumo de recursos por tareas en segundo plano.
Trazabilidad	SLAs de niveles de servicio, requerimientos de auditoría, MonitoreoService.

11. Bitácora de IA

Iteración	1
Objetivo	Determinar la información necesaria para construir el diagrama de clases de nuestra empresa de paquetería
Prompt	¿Qué datos necesitas para crear un diagrama de clases UML de una empresa dedicada a la entrega de paquetes? Establece las clases y relaciones para el diagrama.
Respuesta útil	Se identificaron las clases, atributos, métodos, relaciones, multiplicidades y reglas de negocio como elementos necesarios para el modelo.
Error o supuesto	La propuesta inicial del diagrama UML era general y no estaba adaptada.
Decisión	Realizamos una lluvia de ideas para proponer las entidades y relaciones, después cercioramos nuestra propuesta en el siguiente prompt.

Iteración	2
Objetivo	Establecer las entidades y reglas de negocio de CargoBox.
Prompt	Establece las clases y relaciones para el diagrama.
Respuesta útil	Se propusieron entidades como Cliente, Envío, Paquete, Pago, Seguimiento, Tarifa, Repartidor, Vehículo y Sucursal, junto con reglas relacionadas con pagos, capacidad y entregas.
Error o supuesto	Algunas relaciones y multiplicidades eran propuestas iniciales y requerían validación según las operaciones reales del modelo.
Decisión	Se utilizó la propuesta como base conceptual y se elaboró un diagrama de clases con las entidades consideradas necesarias.

Iteración	**3**
Objetivo	Establecer las entidades y reglas de negocio de CargoBox.
Prompt	Establece las clases y relaciones para el diagrama.
Respuesta útil	Se propusieron entidades como Cliente, Envío, Paquete, Pago,

	Seguimiento, Tarifa, Repartidor, Vehículo y Sucursal, junto con reglas relacionadas con pagos, capacidad y entregas.
Error o supuesto	Algunas relaciones y multiplicidades eran propuestas iniciales y requerían validación según las operaciones reales del modelo.
Decisión	Se utilizó la propuesta como base conceptual y se elaboró un diagrama de clases con las entidades consideradas necesarias.

Iteración	**4**
Objetivo	Elegir dos procesos relevantes para elaborar diagramas de secuencia.
Prompt	¿Qué flujos complejos conviene elegir para realizar dos diagramas de secuencia de la empresa?
Respuesta útil	Se propuso el registro y creación de un envío, así como el transporte, seguimiento y entrega. También se identificaron escenarios alternativos mediante fragmentos UML.
Error o supuesto	La propuesta inicial incluía más participantes de los que el equipo deseaba utilizar en el modelo.
Decisión	Se seleccionaron ambos procesos porque representan etapas importantes del ciclo de vida de un envío.

Iteración	**5**
Objetivo	Reducir los participantes de los diagramas de secuencia a cuatro.
Prompt	¿Es viable juntar la interfaz, el servicio de envíos, las tarifas, los pagos y la base de datos en una sola clase?
Respuesta útil	Se explicó que es posible centralizar las operaciones en una clase Sistema para simplificar el modelo conceptual, aunque esto concentra responsabilidades.
Error o supuesto	La simplificación conceptual con motivos de realizar el diagrama secuencial no implica que una sola clase deba ejecutar todas las funciones en producción.
Decisión	Se decidió utilizar Cliente, Sistema, Envío y Repartidor como los cuatro participantes principales.

Iteración	6
Objetivo	Representar los dos procesos seleccionados en diagramas de secuencia utilizando los cuatro participantes.
Prompt	Construye ambos diagramas de secuencia con cuatro clases y proporciona el código PlantUML.
Respuesta útil	Se generó código PlantUML para el registro y creación de envíos, y para el transporte, seguimiento y entrega, incluyendo condiciones alternativas y actualizaciones repetitivas.
Error o supuesto	La versión simplificada concentra las operaciones de tarifas, pagos y seguimiento en Sistema. Además, debe distinguirse el objeto Envio de la mercancía física, que en el diagrama de clases está representada por Paquete.
Decisión	Se decidió utilizar los diagramas como representación de alto nivel, revisando la correspondencia entre sus participantes y el modelo de clases.

Iteración	7
Objetivo	Documentar formalmente las decisiones relacionadas con ambos procesos.
Prompt	¿Cómo documentar los procesos y las decisiones arquitectónicas mediante la estructura del ADR?
Respuesta útil	Se propuso una estructura con contexto, decisión, alternativas, justificación, consecuencias y artefactos relacionados.
Error o supuesto	Algunos requisitos, como la cantidad de intentos de pago, la asignación de vehículos y el tratamiento de incidencias, todavía eran supuestos del modelo. El estado de las decisiones también debía ser confirmado por el equipo.
Decisión	Se adoptó una estructura uniforme para documentar cada decisión y justificar las alternativas consideradas. Se adoptó la estructura propuesta como base documental, manteniendo las decisiones como propuestas hasta que el equipo las validara.

12. Validar la propuesta

Validación	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	**Sí.** Todos los agregados, servicios y casos de uso están vinculados directamente con las necesidades (NB-01 a NB-08), reglas de negocio (RN-01 a RN-07) y las historias de usuario del BRD
Lenguaje	¿Los mismos términos significan lo mismo en BRD, UML, código y ADR?	**Sí.** Se mantiene un Lenguaje Ubicuo estricto en todo el proyecto; términos como *Guía Única*, *Evidencia POD*, *Intento de Entrega* y los 7 estados operativos se usan idénticamente en la documentación, diagramas y diseño.
Responsabilidad	¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación?	**Sí.** Toda la lógica de negocio, validaciones físicas (peso $\leq$ 30 kg, dimensiones $\leq$ 1.5 m) y protección de invariantes residen exclusivamente en los agregados (Paquete, Incidencia, Traslado, Entrega) y servicios de dominio.
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	**Sí.** Los componentes reflejados en el mapa de dominio y el diagrama de clases se corresponden de manera coherente con los flujos coordinados por la Capa de Aplicación (*Service Layer*) y los *Handlers*.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	**Sí.** El dominio y la aplicación dependen únicamente de abstracciones e interfaces de repositorios (IPaqueteRepositorio, etc.), desacoplando por completo los detalles de base de datos.
Transacción	¿El límite de la operación y su manejo de error están definidos?	**Sí.** Se utiliza una estrategia de *Unit of Work* que define claramente el inicio y fin transaccional de los comandos, manejando los

		errores de negocio sin alterar estados inconsistentes.
IA	¿Cada aporte de IA fue revisado y su uso está registrado?	**Sí.** Cada sugerencia generada fue analizada, corregida y documentada formalmente en la bitácora de iteraciones del equipo, manteniendo la autoría y validación humana.

13. Repositorio de GitHub

https://github.com/dnmonnav-eng/CargoBox.git

Conclusiones

Pablo Sandoval

La elaboración del diagrama de clases permite identificar las entidades principales del sistema, sus atributos y las relaciones que existen entre ellas. Representan los elementos esenciales de las operaciones y muestran cómo se relacionan los aspectos administrativos, económicos y logísticos del negocio. Los diagramas de secuencia permiten representar la interacción entre los participantes durante procesos específicos, como el registro de un envío, la validación del pago, la asignación de un repartidor y la confirmación de una entrega. Visualizamos el orden de las operaciones y contemplar situaciones alternativas, como los pagos rechazados, los datos inválidos o las entregas no realizadas.

Danna Montoya

En primer lugar, la adopción del patrón Domain Model articulado alrededor del paquete y el envío como agregados principales permitió aislar y proteger las reglas de negocio esenciales, tales como los límites físicos de peso y dimensiones, la parametrización de tarifas y la toma de evidencia digital de recepción. Al encapsular esta lógica dentro del núcleo del dominio, se evita que las capas externas asuman validaciones indebidas, garantizando la integridad de las operaciones logísticas y la mantenibilidad del código a largo plazo. Por otra parte, la estructuración de una capa de aplicación organizada en cinco servicios especializados facilitó una clara separación de responsabilidades, donde cada servicio orquesta flujos específicos de recepción, distribución, entrega, incidencias y monitoreo sin contaminarse con lógica operativa. La integración complementaria de patrones de persistencia como Repository y Unit of Work, sumada a la separación de consultas mediante un enfoque CQRS básico, otorga al sistema la capacidad de escalar de forma eficiente, soportar procesos automatizados en segundo plano y responder con agilidad ante escenarios de alta demanda.

Carlos Oyervides

En este trabajo vi cómo se pasa de elegir un patrón de organización de la lógica a bajarlo hasta la capa de servicios. Primero comparé Transaction Script, Table Module y Domain Model. Entendí que Transaction Script funciona bien cuando las reglas son simples, porque hay un procedimiento por cada caso de uso, pero mezcla la lógica con la persistencia y se vuelve difícil de mantener cuando el sistema crece Lo que más me dejó este trabajo es que las cuatro partes se

conectan. La elección del patrón justifica el modelo de dominio, y el modelo define lo que el service layer puede hacer. Por eso creo que Domain Model fue la decisión correcta para CargoBox: si cambia una regla, se corrige en un solo lugar y los servicios no se tocan.